Assignment 3: GRPO

Introduction

In this assignment you will implement a minimal, end-to-end reinforcement-learning pipeline: Group Relative Policy Optimization (`GRPO`) on the Countdown task (Section 1.1). You will build the answer parsers and the reward function (Sections 1.2 and 1.3), compute group-normalized advantages and per-token losses (Section 1.4), and train Qwen3-1.7B with them. You will also implement the `Dr.GRPO` variant. In Problem 1 you run the experiments, report accuracy and reward trends, and analyze successes and failures. Problem 2 (Section 2) is an optional, open-ended extension for extra credit.

To complete the assignment, you will implement Tasks 1–5 in `starter.py`, answer the two conceptual questions in Section 1.4, and run the zero-shot baselines and the main experiments in Section 1.6.

What to submit.

- `starter.py` with Tasks 1–5 implemented.
- A PDF report with your answers to the conceptual questions, the zero-shot baselines, the table and plot of the final task accuracy of GRPO and Dr.GRPO trained at L_{256} and L_{512}, and your analysis of successful and failed cases. Include a link to your model checkpoints.
- Your Problem 2 investigation for extra credit, as a separate section of the report.

Setup instructions are in the repository's `README.md`, and recommended settings for each GPU type are in Appendix A.

Contents

1 GRPO for Countdown Math Problem **2**
- 1.1 Introduction and Experimental Setting . . . 2
- 1.2 Task 1: Verifier Functions . . . 3
- 1.3 Task 2: Reward Function . . . 4
- 1.4 GRPO: Group Relative Policy Optimization . . . 4
  - 1.4.1 Task 3: `compute_group_normalized_advantages` . . . 6
  - 1.4.2 Task 4: `compute_loss` . . . 6
  - 1.4.3 Task 5: `masked_mean` and `masked_mean_drgrpo` reductions . . . 8
- 1.5 Putting It Together: Training Loop, Grouping, and Microbatches . . . 9
- 1.6 Experiments . . . 11
  - 1.6.1 Zero-Shot Baseline . . . 11
  - 1.6.2 Main Experiments . . . 11

2 Optional Extra Credit: Problem 2 - Open-Ended Investigation **11**

A Recommended GPU Settings for the Rollout Engine **12**

1 GRPO for Countdown Math Problem

For this problem, we will implement a **Group Relative Policy Optimization (GRPO)** training pipeline. This includes setting up the experimental environment, designing a reward function, implementing the core GRPO algorithm, and exploring its variant **Dr.GRPO**.

1.1 Introduction and Experimental Setting

Countdown Problem. The Countdown problem is a numerical reasoning task. Given a finite set of positive integers $S = \{n_1, n_2, \ldots, n_k\}$ and a target positive integer T, the language model must generate an expression E that evaluates to T. In our experiments, the size of the input number set is constrained to either three or four numbers ($k \in \{3, 4\}$).

The equation E must satisfy the following constraints:

1. **Operator constraint:** only the four basic arithmetic operations $+, -, *, /$ and parentheses are allowed.
2. **Usage constraint:** each number $n_i \in S$ may be used at most once. (The original game asks for every number to be used exactly once; the verifier in this assignment also accepts equations that use a subset, see Task 1.)

The model's output is the final equation.

```
Example #1
Input: [7, 7, 65]
Target: 51
Expected Output: (65 - 7) - 7

Example #2
Input: [34, 84, 83, 72]
Target: 39
Expected Output: (72 - 34) + (84 - 83)

Example #3
Input: [9, 4, 3, 7]
Target: 101
Expected Output: (9 * (4 * 3)) - 7
```

Experimental setting. We use the following model, dataset, rollout engine and hardware:

- **Model:** `Qwen/Qwen3-1.7B`
- **Dataset:** justinphan3110/Countdown-Tasks-3to4
- **Rollout engine:** a small vLLM-style inference engine written in PyTorch (FlexAttention, paged KV cache, CUDA-graph decoding). It is provided as a single self-contained file, `flex_engine.py`, which you do not need to modify.
- **Compute:** one GPU with at least 40 GB of memory (A100 40/80 GB, H100, H200 or RTX PRO 6000). Recommended engine settings for each GPU are applied automatically and listed in Appendix A.

Prompt. Every training and evaluation prompt uses the template below (the `TEMPLATE` string in `starter.py`), wrapped in Qwen3's chat template with a short system prompt. Qwen3 then reasons inside its own `<think>` block before writing the answer.

```
Using the numbers {numbers}, create an equation that equals {target}.
You can use basic arithmetic operations (+, -, *, /) and each number can only be used once.
Show your reasoning in <think> </think> tags. And return the final equation in <answer> </answer> tags. Keep your reasoning under {max_tokens} tokens.
For example, numbers = [1, 2, 3, 4] and target = 5, the answer is <answer>(1 + 2) * 3 - 4</answer>.
```

1.2 Task 1: Verifier Functions

Given a generation $\mathcal{G}$ from our model M, we define verifier functions to extract and validate the final equation E.

`_extract_answer` Extract the content from the last occurrence of the `<answer>...</answer>` tag in the model's rollout generation $\mathcal{G}$.

```python
def _extract_answer(solution_str: str) -> str | None:
    ### YOUR CODE HERE ###
    ...

# Input/Output Format:
rollout_response = "Okay, let's see. I need to use 79, 17, ... So the equation is <answer>(79 - (60 - 17))</answer>"
extracted_answer = _extract_answer(rollout_response)
assert extracted_answer == "(79 - (60 - 17))"
```

`_validate_numbers` Check that the numbers used in E form a valid **subset** of the available numbers S, each used at most once (a number listed twice in S may be used twice).

```python
def _validate_numbers(equation_str: str, available_numbers: List[int]) -> bool:
    ### YOUR CODE HERE ###
    ...

# Input/Output Format:
# Example 1: Valid Usage (subset)
assert _validate_numbers("(64 + 14) - 16", [64, 14, 16, 7]) == True

# Example 2: Invalid Usage (number used too many times)
assert _validate_numbers("(7 + 7) + 7", [7, 7, 65]) == False

# Example 3: Invalid Usage (number not available)
assert _validate_numbers("10 + 9", [10, 8, 2]) == False
```

`_evaluate_equation(equation_str)` Evaluate E, allowing only digits, spaces and the characters `+ - * / ( )`. You may use Python's `eval`, but return `None` for anything invalid or unsafe:

other characters (letters, Unicode operators such as × or ÷, =), implicit multiplication such as `2(3)`, division by zero, or syntax errors.

```python
def _evaluate_equation(equation_str: str) -> float | None:
    ### YOUR CODE HERE ###
    ...

# Input/Output Format:
assert _evaluate_equation("(65 - 7) - 7") == 51.0
```

1.3 Task 2: Reward Function

`reward_fn` The reward $R(\mathcal{G})$ is calculated based on the validity of the extracted equation E and its result $\mathcal{R}$ compared to the target T. This definition directly maps to the reward structure defined in the function's docstring:

- $R(\mathcal{G}) = \mathbf{1.0}$: Assigned if the equation E is **Correct** (correct equation with correct answer).
- $R(\mathcal{G}) = \mathbf{0.1}$: Assigned if the generation makes an attempt: it has an `<answer>` tag, but the equation is wrong (wrong value, wrong numbers, or invalid).
- $R(\mathcal{G}) = \mathbf{0.0}$: Assigned if no `<answer>` tag is found in the generation $\mathcal{G}$.

```python
def reward_fn(generated_text: str, ground_truth: Dict) -> float:
    """
    Reward function for countdown math problems:
    - 1.0: Correct equation with right answer
    - 0.1: Valid format but wrong answer, or wrong numbers used, or invalid equation
    - 0.0: No answer tag found (completely failed format)
    """
    # Implements the reward logic using the verifier functions:
    # _extract_answer, _validate_numbers, and _evaluate_equation.

numbers = [79, 17, 60]
target = 36
ground_truth = {"numbers": numbers, "target": target}

# Example 1: Correct Answer
rollout = "...thinking... <answer>(79 - (60 - 17))</answer>"
assert reward_fn(rollout, ground_truth) == 1.0



# Example 2: Correct usage, Wrong Answer (Partial, R = 0.1)
rollout = "...thinking...<answer>(79 + 60) + 17</answer>"
assert reward_fn(rollout, ground_truth) == 0.1  # valid format of <answer>

# Example 3: No Answer Tag (Failed Format, R = 0.0)
rollout = "...thinking...The final answer is 62."
assert reward_fn(rollout, ground_truth) == 0.0
```

1.4 GRPO: Group Relative Policy Optimization

Paper: DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Background: From REINFORCE to GRPO Teaching LLMs to solve mathematical problems through reinforcement learning presents unique challenges. While Supervised Fine-Tuning (SFT) requires expensive human-annotated step by step solutions, RL allows models to learn from trial and error with automatic verification. In this framework, the language model serves as a policy π_θ that, given a mathematical question q, generates a complete solution o, which receives a scalar reward $R(q, o)$ based on correctness.

The classical REINFORCE algorithm optimizes the policy using the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{o\sim\pi_\theta(\cdot|q)} \left[R(q, o) \cdot \nabla_\theta \log \pi_\theta(o|q) \right]$$

Directly weighting gradients by raw rewards can be noisy, to address this, it is common to introduce a baseline $b(q)$ and use advantages instead:

$$\nabla_\theta J(\theta) = \mathbb{E}_{o\sim\pi_\theta(\cdot|q)} \left[A(q, o) \cdot \nabla_\theta \log \pi_\theta(o|q) \right]$$

where $A(q, o) = R(q, o) - b(q)$ is the advantage function that measures how much better or worse this response is compared to the baseline.

GRPO: Group Relative Policy Optimization Traditional approaches require training a separate value function $V_\phi(q)$ to serve as the baseline, which introduces additional complexity to the training pipeline. To address this, GRPO elegantly simplifies the need for a value function through group sampling. For each question q, the policy samples a group of G responses $\{o^{(i)}\}_{i=1}^{G} \sim \pi_\theta(\cdot|q)$. Each response receives its reward $r^{(i)} = R(q, o^{(i)})$, and the group itself provides a natural baseline through its statistics.

The group-normalized advantage for the i-th response in GRPO is computed as:

$$A^{(i)} = \frac{r^{(i)} - \text{mean}(r^{(1)}, \ldots, r^{(G)})}{\text{std}(r^{(1)}, \ldots, r^{(G)}) + \texttt{advantage_eps}} \tag{1}$$

Here, `advantage_eps` is a small constant added to the denominator to prevent division by zero when all responses in a group receive identical rewards.

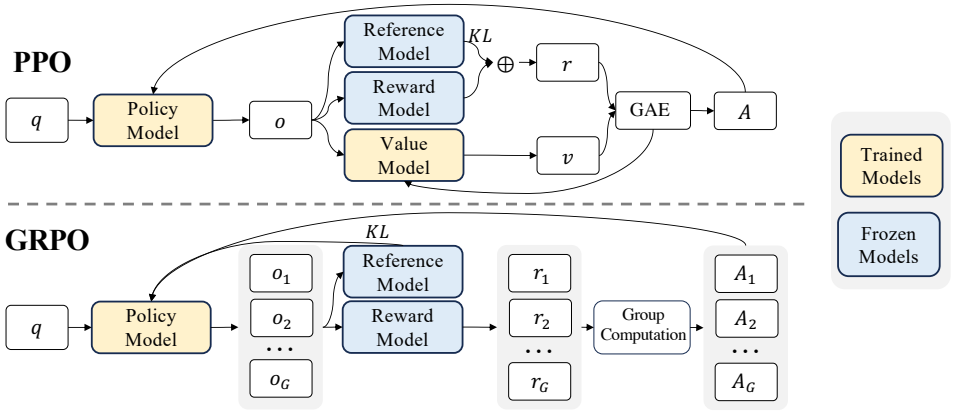


Figure 1: GRPO eliminates the need for a separate value function by using group sampling to estimate baselines.

1.4.1 Task 3: `compute_group_normalized_advantages`

Your task is to implement a function that takes the raw rewards from a batch of rollouts and transforms them into normalized advantages.

```python
def compute_group_normalized_advantages(
    rollout_responses: List[str],
    repeated_ground_truths: List[Dict],
    reward_fn: Callable[[str, Dict], float],
    group_size: int,
    advantage_eps: float,
    normalize_by_std: bool,
) -> Tuple[torch.Tensor, torch.Tensor, Dict[str, torch.Tensor]]:
    """
    Computes advantages by normalizing rewards within groups.

    Args:
        rollout_responses: List of generated responses (length = batch_size * group_size)
        repeated_ground_truths: Ground truth repeated for each response
        reward_fn: Function to compute rewards
        group_size: Number of responses per question (G)
        advantage_eps: Small constant for numerical stability (epsilon)
        normalize_by_std: If True, normalize by std (GRPO); if False, don't (DR-GRPO)

    Returns:
        advantages: Flattened tensor of advantages (shape: [batch_size * group_size])
        raw_rewards: Original rewards before normalization
        metadata: Dictionary with 'mean', 'std', 'max', 'min' of raw rewards
    """
    ### YOUR CODE HERE ###
    pass
```

Implementation Notes: When `normalize_by_std` is True, the advantages are computed as shown in the Eq. (1). If `normalize_by_std` is False (as in the Dr.GRPO paper), the advantages are simply calculated as the difference between each reward and the group mean, without normalizing by the standard deviation.

1.4.2 Task 4: `compute_loss`

After computing advantages, we construct a per-token loss by decomposing the conditional log-probability of the whole response. Using the chain rule, the policy factorizes as

$$\pi_\theta(o \mid q) = \prod_{t=1}^{|o|} \pi_\theta\big(o_t \mid o_{<t}, q\big)\,, \quad \Rightarrow \quad \log \pi_\theta(o \mid q) = \sum_{t=1}^{|o|} \log \pi_\theta\big(o_t \mid o_{<t}, q\big).$$

This means that maximizing the sequence log-likelihood is equivalent to maximizing the sum of per-token log-likelihoods. In GRPO, each sampled response for a prompt receives a single scalar advantage $A^{(i)}$; we broadcast it to all tokens in that response ($A_t = A^{(i)}$ for all t). The PPO-style sequence loss then sums token losses:

$$\mathcal{L}_{\text{seq}} = -\sum_{t=1}^{|o|} \min(A_t \,\text{ratio}_t,\ A_t \,\text{clip}(\text{ratio}_t, 1-\epsilon, 1+\epsilon))\,.$$

Where the probability ratio compares the new and old policies at each token:

$$\text{ratio}_t = \frac{\pi_\theta(o_t \mid o_{<t}, q)}{\pi_{\theta_{\text{old}}}(o_t \mid o_{<t}, q)} = \exp\left(\log \pi_\theta - \log \pi_{\theta_{\text{old}}}\right),$$

where $\pi_{\theta_{\text{old}}}$ is the pre-update policy or the policy that was used to generate the rollout. Following PPO, the clipping simply keeps the new policy from drifting too far from the current one, which stabilizes training even when advantages are large.

```python
def compute_loss(
    advantages: torch.Tensor,        # [B, 1]
    policy_log_probs: torch.Tensor,  # [B, T]
    old_log_probs: torch.Tensor,     # [B, T]
    clip_range: float,
) -> Tuple[torch.Tensor, Dict[str, torch.Tensor]]:
    """
    Per-token PPO clipped surrogate loss.

    Steps:
    1. ratio_t = exp(policy_log_probs - old_log_probs)
    2. unclipped term: A_t * ratio_t
    3. clipped term: A_t * clip(ratio_t, 1 - clip_range, 1 + clip_range)
    4. loss = -min(unclipped term, clipped term)

    Returns (loss [B, T], metadata), e.g. metadata = {"clip_fraction": ...}.
    """
    ### YOUR CODE HERE ###
```

Implementation details Let batch size be B and let T be the number of tokens in each example (note that the prompt tokens are masked out).

- `policy_log_probs` $\in \mathbb{R}^{B \times T}$: computed by applying `log_softmax` to model logits and gathering at the predicted token indices; one log-prob per token.
- `old_log_probs` $\in \mathbb{R}^{B \times T}$: the same meaning but comes from the pre-update policy in one rollout.
- `advantages` $\in \mathbb{R}^{B}$: one scalar per response; broadcast along the time axis to shape $B \times T$ inside `compute_loss`.
- `response_mask` $\in \{0, 1\}^{B \times T}$: masks out prompt positions so loss is computed only on response tokens.

The `compute_loss` function computes per-token probability ratios, multiplies by advantages, applies elementwise clipping, and returns a loss matrix of shape $B \times T$.

Conceptual: Clipping and Policy Regime

Question.

1. In the PPO-style loss above, is clipping more critical for **on-policy** or **off-policy** updates in this setup? Briefly justify your answer in terms of the probability ratio.
2. In the context of GRPO for this task, list **two advantages of on-policy** and **two advantages of off-policy** and give a brief explanation for each.

1.4.3 Task 5: `masked_mean` and `masked_mean_drgrpo` reductions

With per-token losses $L \in \mathbb{R}^{B \times T}$ computed, we must reduce them to a single scalar for optimization while respecting the response mask M. In this part, we will implement two reduction schemes: (1) the `masked_mean`, which is the original GRPO proposal where the reduction is based on the response length, and (2) `masked_mean_drgrpo`, where the reduction is instead scaled by a constant, specifically the maximum response length, as in Dr.GRPO. Interestingly, the choice of reduction also changes the per-token gradients during backpropagation, since different denominator schemes rescale token losses differently.

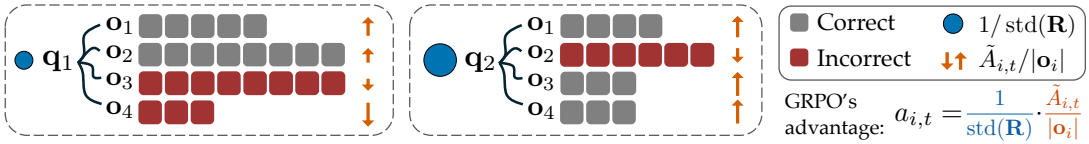


Figure 2: This figure illustrates response-level length bias: dividing by the response length $|o_i|$ skews gradient magnitudes. For correct responses (gray tokens), shorter responses receive larger updates (bigger orange arrows) compared to longer responses. Conversely, for negative advantages from incorrect responses (red tokens), longer responses are penalized less.

`masked_mean` (length-normalized). Average the per-token loss over each sequence's response tokens and then average across the batch.

```
def masked_mean(tensor: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    """
    Mean over response tokens per sequence, then mean over batch.

    Args:
        tensor: Per-token loss [B, T]
        mask:  Binary mask   [B, T] (1=response token, 0=prompt/pad)
    Returns:
        Scalar loss.
    """
```

`masked_mean_drgrpo` (constant-normalized). Dr.GRPO removes response-length normalization by replacing the per-sequence denominators with a fixed constant `num_tokens`.

```
def masked_mean_drgrpo(tensor: torch.Tensor, mask: torch.Tensor, num_tokens: int) ->
↪ torch.Tensor:
    """
    Dr.GRPO reduction: sum over response tokens, divide by (num_tokens * B).

    Args:
        tensor: Per-token loss [B, T]
        mask:  Binary mask [B, T]
        num_tokens: A fixed constant
    Returns:
        Scalar loss.
    """
    ### YOUR CODE HERE ###
```

```
    pass
    ### END YOUR CODE ###
```

Both reductions use the mask to only calculate the loss on the response tokens. The difference is that one normalizes by the response length for each sequence, while the other uses a fixed constant.

Per-token gradients under two reductions

Question. Consider a batch with two responses: one short (3 response tokens and 3 masked tokens) and one long (6 response tokens and 0 masked tokens). Assume similar per-token log-prob ratio and advantages, and set the fixed maximum response length to 6 for the `masked_mean_drgrpo` reduction.

1. Apply each reduction to the batch and call `backward()`, then compare the gradient w.r.t the per-token log-prob ratio. What do you observe?
2. Implications: Give an example scenario where `masked_mean` is preferable to `masked_mean_drgrpo`, and one where the opposite is true. Give a brief explanation for each.

1.5 Putting It Together: Training Loop, Grouping, and Microbatches

This section is a short reading that explains how the whole pipeline fits together in practice.

One rollout-training step (high level). Each training step follows the same pattern:

1. **Sample prompts.** Pick a small set of prompts from the training set. We intend to produce multiple responses for each prompt in order to compare them within a group.
2. **Duplicate per group and generate.** For each prompt, duplicate it `group_size` times and send these copies to the rollout engine. This yields one response per copy. Duplicating is what forms a *group* of responses that all answer the same prompt. The flex engine runs the policy's own modules, so every rollout uses the latest weights and no weight sync is needed. (With a separate engine such as vLLM, you would copy the weights into the engine before every rollout.)
3. **Score and compute advantages.** Use the `reward_fn` to score each response. Reshape the rewards into groups, subtract each group's mean to get relative scores, and (optionally) divide by the group's standard deviation. This produces one advantage value per response. Importantly, advantages are computed *before* any microbatching so that all tokens of a response share the same scalar advantage.
4. **Tokenize and mask.** Build `input_ids`, `labels`, and a `response_mask` that zeros out prompt tokens so the loss is only computed on generated tokens.
5. **Microbatch and accumulate gradients.** Split the rollout batch into several microbatches to fit in memory. For each microbatch:
   - Compute per-token log-probabilities of the policy.
   - Broadcast the response-level advantage across its tokens.
   - Form the PPO-style clipped per-token loss using the probability ratio.
   - Reduce to a scalar loss with either `masked_mean` or `masked_mean_drgrpo`.

- Divide the scalar loss by the number of microbatches (i.e., gradient accumulation steps) and call `backward()` to accumulate gradients.

6. **Step optimizer and scheduler.** After all microbatches are processed, clip gradient norm, then call `optimizer.step()` and `scheduler.step()`. Gradients are zeroed *once* per rollout step, not per microbatch.

7. **Evaluate and log periodically.** Every few steps, generate on a held-out set, compute accuracy and reward statistics, and log short examples for quick inspection.

Why a rollout engine instead of `model.generate`? Rollouts dominate the cost of RL training: every step generates 128 responses of up to L tokens. An inference engine is much faster than `model.generate` for this. It keeps a *paged* KV cache, so memory is allocated per 128-token page as sequences grow. It uses *continuous batching*, so a finished sequence immediately frees its slot. And it replays decoding steps as CUDA graphs. `flex_engine.py` does all three in about 500 lines of PyTorch, using FlexAttention for attention over the paged cache. Because it shares the policy's weights, training and generation use one copy of the model on one GPU. The settings that trade memory for speed (KV-cache size, concurrent sequences, attention tile sizes, microbatch size) are discussed in Appendix A.

Why microbatches? Large rollout batches often do not fit into GPU memory. Microbatching keeps memory usage manageable while preserving the effect of a larger batch: each microbatch adds to the same gradient buffers, and we scale the loss so that the accumulated gradient matches what a single big batch would produce (up to small numerical differences).

Takeaway. Think of the pipeline as: sample prompts → duplicate to form groups → generate/rollout → score & compute advantages → microbatch forward/loss/backward with accumulation → optimizer step & periodic eval. Once this flow is clear, switching between GRPO and Dr.GRPO is mostly about whether you normalize advantages by the group std and which reduction you use for the per-token losses.

1.6 Experiments

1.6.1 Zero-Shot Baseline

Before running the GRPO pipeline, check how well `Qwen/Qwen3-1.7B` does zero-shot (prompting only). Once Tasks 1–2 are done, run `python zero_shot_eval.py --max_tokens L` for the three generation budgets L_{256}, L_{512} and L_{1024}. You should get roughly:

- $L_{256} \approx 0.1$–0.2%: the model almost never finishes thinking within 256 tokens
- $L_{512} \approx 15\%$
- $L_{1024} \approx 38\%$

Accuracy is measured with one sample per problem at temperature 1, so expect about ± 1 point of sampling noise; `--n_samples 4` averages 4 samples per problem.

Main Experiments

1.6.2 Main Experiments

For our upcoming RL experiments, our target is to train the `Qwen/Qwen3-1.7B` model at L_{256} and L_{512} to outperform zero-shot performance obtained at L_{1024} ($\approx 37\%$).
Using your implemented GRPO and Dr.GRPO pipeline, run a full training loop (80 steps by default) for models with maximum generation lengths of $\mathbf{L}_{256}$ and $\mathbf{L}_{512}$, for example `python starter.py --loss_type grpo --max_tokens 256` (also `--loss_type dr_grpo` and `--max_tokens 512`). An 80-step run takes roughly 10–40 minutes depending on the GPU and budget (Appendix A).
Experimental Setup:

1. Train with a maximum generation length of L_{256}.
2. Train with a maximum generation length of L_{512}.

Report performance: tabulate and plot the final task accuracy of the models trained at L_{256} and L_{512} (GRPO and Dr.GRPO), and compare them with the zero-shot baselines. Training curves are logged to TensorBoard and to `output/tb/<run>/<timestamp>/{train,eval}_metrics.jsonl`.
Analysis: write a short analysis of the results, including an interpretation of successful and failed cases of the trained models. A hypothesis (or an implementation) of what could still be improved in the training setup or the reward function is highly recommended.
Practical notes.

- At L_{256} almost every early rollout is cut off before `<answer>`, so all rewards (and the gradient) are zero for roughly the first 10–20 steps. This is expected; learning starts from the first rare successes.
- Identical runs with different seeds can differ by about 2 points at L_{512} (more at L_{256}). Compare configurations over seeds rather than single runs when the differences are small.

2 Optional Extra Credit: Problem 2 - Open-Ended Investigation

This optional problem asks you to investigate the RL training pipeline more deeply. The goal is to show strong problem solving and analysis: **your analysis of the approach, whether**

it succeeds or fails, is the most important part.

Some possible starting points:

- **Improve the current results:** push accuracy further, for example with longer training or better settings, and explain what helped.
- **Reward shaping:** is the 0.1 reward for a wrong answer necessary? Train with 0 instead and compare GRPO and Dr.GRPO at L_{256} and L_{512} over several seeds. Its effect depends on how the advantages are normalized, so explain why.
- **Reasoning under a tight budget:** at L_{256} most failures are responses cut off before `<answer>`. Study ways to reason within the budget: Qwen3's non-thinking mode, forcing an answer when the budget runs out, length-aware rewards, or a curriculum from L_{512} down to L_{256}.
- **Adaptation:** apply the GRPO/Dr.GRPO pipeline to a similar verifiable reasoning task. Be creative.
- ...

Acknowledgments

We are grateful to TinyZero for the Countdown task, which provides an approachable introduction to RLVR, and to the Stanford CS336 staff for Assignment 5, which served as a basis for this project. The rollout engine builds on Jonathan Chang's flex-nano-vllm (MIT License).

A Recommended GPU Settings for the Rollout Engine

Training and generation share one GPU. Memory is split between the policy (weights, gradients and AdamW state, about 14 GB), the activations of one training microbatch, and the engine's KV cache. `starter.py` detects the GPU and applies the preset at the end of `flex_engine.py`; any of the flags below overrides it.

- `--kv_cache_tokens`: KV-cache capacity. Each token costs 28 layers $\times$ 8 KV heads $\times$ 128 $\times$ 2 $\times$ 2 bytes $\approx$ 0.11 MB. One 128-rollout batch needs about 72k tokens at L_{256} and 100k at L_{512}, in 128-token pages. A smaller cache still works, but sequences are paused and resumed, which is slower.
- `--max_num_seqs`: sequences decoded together. 512 instead of 256 makes the 1024-problem evaluation about 15% faster when the cache can hold them.
- `--block_m`, `--block_n`: FlexAttention tile sizes for decode. 16×64 is the fastest on every GPU tested: 1.47–1.56$\times$ faster than 32×32 on A100, H100 and H200, and 1.08$\times$ on RTX PRO 6000. PyTorch's own decode defaults are 28–40% slower and do not fit the RTX PRO 6000's 101 KB of shared memory. Prefill uses PyTorch's defaults, which are 3–7% faster. Attention is about 35% of decode time.
- `--grad_acc_steps`: microbatches per 128-rollout step (32 means microbatch 4). This changes only speed and memory, not the gradient. Microbatches larger than 4 are only 3–6% faster but need much more memory; 64 (microbatch 2) frees about 8 GiB on 40 GB cards.

GPU	L	KV tokens	seqs	grad_acc	peak mem.	s/step (gen + train)	100 steps
A100 40GB	256	100k	256	32	34.6 GiB	5.0 + 5.3	∼25 min
A100 40GB	512	100k	256	**64**	32.7 GiB	9.9 + 9.8	∼47 min
A100 80GB	256	270k	512	32	52.6 GiB	4.6 + 4.9	∼21 min
A100 80GB	512	400k	512	32	73.1 GiB	8.9 + 8.1	∼37 min
H100 80GB	256	270k	512	32	53.8 GiB	3.6 + 3.2	∼16 min
H100 80GB	512	400k	512	32	73.4 GiB	7.1 + 5.1	∼27 min
H200	256	270k	512	32	53.8 GiB	2.8 + 2.3	∼12 min
H200	512	400k	512	32	73.5 GiB	5.5 + 3.4	∼20 min
RTX PRO 6000	256	270k	512	32	52.9 GiB	3.6 + 3.9	∼17 min
RTX PRO 6000	512	400k	512	32	≈73 GiB[†]	7.3 + 5.2	∼31 min

Table 1: Presets (all use 16×64 tiles) with the peak GPU memory and time per step measured in 12-step training runs. "100 steps" includes 11 evaluations on the full test set (one every 10 steps); an 80-step run takes about 80% of it. A100 and H100 numbers are from the PCIe versions. [†]Speed measured with a 200k cache and 256 sequences (peak 51.5 GiB); the preset's larger cache uses about 73 GiB, as on the A100 80GB.

Running on a Slurm cluster. Request one GPU of a supported type and run from the repository root, for example `LOSS=grpo MAXTOK=256 sbatch slurm/train.sbatch`. The job script asks for `--gres=gpu:1` with a constraint on the GPU type; the preset adapts to whichever GPU the job gets.